Lab 4 AVCV 2026/2027: Fire-risk classification from satellite images

CUSTOM and FINE-TUNING networks on the FireRisk subset

Daniel Truzman (100475161) César Álvarez-Cascos (100475191) • CodaBench team `100475161_100475191`

1. INTRODUCTION

The goal of this lab is to assign one of seven FireRisk classes (Water, Very Low, Low, Moderate, High, Very High and Non-burnable) to a single RGB satellite image. It is worth stressing that this is an image-classification problem and not fire detection: the label describes the fire risk of the area shown, not whether a fire is visible in the image.

The task is harder than it first appears for two reasons. The classes are strongly imbalanced, from 2.4% of the images for Water to 31% for Very Low, so a model that ignores the rare classes can still look accurate; this is why macro-F1, which weights every class equally, is the metric of the lab. In addition, the five risk levels are ordered and look alike, and, as the results will show, neighbouring levels such as Low, Moderate and High are the main source of errors.

We work with a subset of FireRisk [1] that contains 5,861 images of 320×320 pixels. It is split in a stratified way into 3,516 training, 1,172 validation and 1,173 test images, whose labels are hidden; Table 3 gives the class counts. Note that the model input is smaller than the original image: both final networks receive 224×224 crops (Section 2.1).

We developed the two systems required by the lab. The CUSTOM system is a VGG-style CNN trained from scratch, whose design was refined through a staged random search. The FINE-TUNING (FT) system adapts ImageNet-pretrained torchvision models, and Swin-T was finally selected among them. Beyond the final models, the experiments tried to answer a few concrete questions: whether the network from scratch was limited by capacity or by the data, how class weighting and augmentation affect the minority classes, and whether ensembles or retraining on train+val would help.

2. METHODOLOGY

2.1. Preprocessing and data augmentation

Images are read as floating-point values in $[0, 1]$ and normalised with the ImageNet mean and standard deviation. This normalisation is required by the pretrained weights, and we kept it for CUSTOM so that both systems share one pipeline. During training we take a random 224×224 crop of each 320×320 image and then augment every batch on the GPU with horizontal and vertical flips, 90° rotations and random changes of gamma, brightness, contrast and saturation, using either a *light* or a *strong* preset. Validation and test images are only centre-cropped to 224×224, without augmentation or test-time augmentation (TTA).

These augmentations suit satellite imagery: aerial views have no canonical orientation, so flips and rotations create new samples without changing the label, while the colour changes are kept mild so that vegetation cues are preserved. The search selected the *strong* preset for CUSTOM and the *light* one for Swin-T. We also tried mixup [2], CutMix [3], TTA and evaluation at the full 320 px, but none of them gained more than the seed noise of ± 0.008 (Table 2).

2.2. System design: CUSTOM network

The CUSTOM network (Table 5) is built only from primitive layers. It stacks four VGG-style blocks [4], each made of two Conv 3×3–BatchNorm [5]–ReLU layers followed by 2×2 max-pooling, with 48, 96, 192 and 384 channels. Global average pooling (GAP) [6] then feeds a small classifier, FC(384,256)–ReLU–Dropout(0.3)–FC(256,7). The whole network has 2.74 M parameters.

This design differs substantially from the provided reference, which has three blocks of a single 5×5 convolution with 6, 16 and 32 channels, no normalisation and a Flatten head. We kept each change only if it raised validation macro-F1. The first change was to replace the Flatten head with GAP: with Flatten, the first fully connected layer held about 99% of the parameters, and such heads scored lower. We then used two stacked 3×3 convolutions with BatchNorm in each block, which cover the same receptive field as a 5×5 convolution with fewer weights and make training more stable. Finally, widening the network to 48 base channels added +0.03, whereas going further (64 channels, five blocks or larger crops) did not help.

2.3. System design: FINE-TUNING network

Instead of the provided AlexNet [7], we compared three modern backbones of 24–28 M parameters, all with torchvision ImageNet-1k weights [8]: ConvNeXt-T [9], Swin-T [10] and EfficientNetV2-S [11]. EfficientNetV2-S

produced a non-finite loss in fp16 and was therefore replaced by ResNet-50 [12]. In every model we replaced the classifier with a new `Linear(in, 7)` layer (768→7 for Swin-T) and the adaptive pooling with a deterministic mean.

No layer was frozen, so backbone and head are trained together. To match the pretraining conditions, the input is 224×224 and normalised with the ImageNet statistics. Because the backbone is already trained while the head starts from scratch, we used two learning rates [13]: a small one for the backbone and, for the head, the same rate multiplied by 1 or 10. Biases and normalisation layers have no weight decay, and stochastic depth [14] adds further regularisation. Swin-T (swin_p2_1) obtained the best validation macro-F1 (0.546) and was also the most stable run: its training F1 was 0.77, whereas ResNet-50 reached 0.99.

2.4. Training protocol and model selection

Table 1: Training configuration of the selected models.

	CUSTOM (n07_60ep)	FT (Swin-T, swin_p2_1)
Loss	Cross-entropy, class weights $\propto 1/\sqrt{n_c}$, no label smoothing, no mixup	
Optimiser & schedule	AdamW [15]; linear warm-up + cosine decay [16]; fp16 mixed precision	
Learning rate	$3.17 \cdot 10^{-4}$	backbone $2.37 \cdot 10^{-5}$, head $\times 1$
Weight decay	0.0464	0.0395
Batch / warm-up	128 / 1 epoch	32 / 2 epochs
Epochs (selected epoch)	60 (36)	30 (26)
Augmentation	strong	light, drop path 0.2
Common	Seed 42; full train split (no subsampling); Kaggle Tesla T4; PyTorch 2.11 [17]	

Table 1 summarises the training configuration of both selected models. Their hyperparameters come from a random search [18] with median-stopping pruning [19], organised from coarse to fine. For CUSTOM, phases 1–3 ran 24 trials on the architecture and optimiser, 16 trials on secondary settings and three finalists trained for 40 epochs. All of them plateaued at 0.45–0.465 with almost equal training and validation scores, which indicated underfitting. Phases 4–5 therefore tested targeted changes, and only widening the network and training for 60 epochs gave clear gains. A larger 121-trial search reached 0.524 on validation but only 0.47 on the public test, so we discarded it. For FT, we ran 10 configurations per network for 10 epochs and retrained the best two of each network for 30 epochs.

In every run we keep the weights of the epoch with the best validation macro-F1, and the final runs use no early stopping. Training is fully deterministic: we enable `use_deterministic_algorithms` and fix the seeds of all libraries and DataLoader workers, so that training one epoch twice yields bit-identical weights.

3. RESULTS AND ERROR ANALYSIS

Our main metric is macro-F1, the mean of the seven per-class F1 scores, so that rare classes count as much as frequent ones. Every decision was based on validation F1; training F1 served only as a diagnostic, and the public CodaBench score was never used for selection.

Table 2 summarises the experiments. The reference network reached only about 0.36 macro-F1, and our first version with a GAP head reached 0.440. Both, however, scored almost the same on training and validation data, a clear sign of underfitting. The random search confirmed this, as it plateaued at 0.45–0.465. We therefore increased capacity instead, widening the network to 48 base channels with two convolutions per block, and trained for 60 epochs, which produced the final CUSTOM network (n07_60ep) with 0.502. Several other ideas brought no gain:

- inverse-frequency class weights, which made results worse;
- more capacity (64 channels, five blocks) or larger crops;
- CutMix and longer training (80 epochs);
- ensembles, per-class threshold tuning and retraining on train+val.

Changing only the seed already moves results by ± 0.008, so smaller differences are not meaningful. For FT, every modern backbone beat AlexNet (0.452), and Swin-T was the best with 0.546.

The learning curves of CUSTOM (Figure 3) show training and validation curves that stay together, with validation F1 saturating at 0.49–0.50 after epoch 30. The model therefore does not overfit; it is limited by the data. Swin-T, in contrast, fits the training set better, with a training F1 of 0.77 against 0.546 on validation.

The per-class results (Table 3) show that Water (0.85) and Non-burnable (0.80) are easy to recognise. The difficult classes are the intermediate levels: Low (0.22), Moderate (0.32) and High (0.33) for CUSTOM, and the same three for FT (0.33, 0.38 and 0.41). Precision and recall are close for most classes, with one clear exception: CUSTOM recovers only 19% of the Low images. The confusion matrices (Figure 1) explain why. Most errors happen among the five risk levels (77% of CUSTOM errors and 76% of FT errors), and about half of all errors fall between two adjacent levels (49% and 52%). Low is the clearest case: CUSTOM labels 40% of the Low images as Very Low, and even Swin-T sends 35% of them there. Very Low is in turn often mistaken for Non-burnable (13% and 11%).

The examples in Figure 2 suggest that part of these errors cannot be solved from the image alone. The Low image is mostly open water and both models predict Water, while the Very High image shows pale, almost bare ground that both models read as Very Low. Since the labels come from a much coarser hazard map than the image patch [1], what makes an area risky is often not visible in it. The difficulty does not seem to come from model capacity: even on its own training images, evaluated without augmentation, CUSTOM reaches only 0.53 (Low 0.30), which suggests that the images themselves carry limited information about these classes. Overall, CUSTOM is small (2.7 M parameters), fast and stable, while FT gains +0.04 on validation at a higher compute cost.

On the public leaderboard (Table 4), our final submission, with the two selected models, scores 0.50 for CUSTOM and 0.49 for FT; an earlier 40-epoch version of the widened network (p4_09_B) had reached 0.49. The final CUSTOM network keeps its validation score (0.502), whereas the models chosen after the longest searches lose the most: Swin-T drops from 0.546 to 0.49, and the best model of the 121-trial search from 0.524 to 0.47. This pattern points to selection bias, since the same validation images were reused across hundreds of trials, combined with the noise of a small public subset.

4. CONCLUSIONS

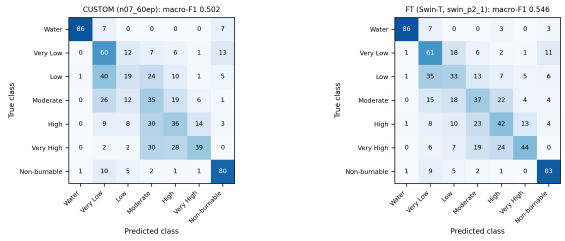
The largest effects came from different sources in the two systems. For CUSTOM, the GAP head, the extra width and the longer schedule together raised validation macro-F1 from 0.44 to 0.50. For FT, the ImageNet pretraining itself was decisive: with a small backbone learning rate, Swin-T reached 0.546 in 30 epochs. Fine-tuning is therefore clearly better on validation, yet both systems score about the same on the public subset (0.50 vs 0.49).

The main limitation lies in the data rather than in the models. The intermediate classes are hard even on training images, and reusing a single validation set for selection inflated the scores of the most searched models by up to about 0.06. For reference, the original FireRisk paper reports only 61–65% accuracy with about 91k images [1]. Realistic next steps are k-fold cross-validation for a more honest selection, losses that exploit the order of the classes, remote-sensing pretraining and a higher input resolution.

Individual contributions.

- **César Álvarez-Cascos (100475191):** design of CUSTOM search phases 1–3 (two-machine random search, finalists and the 121-trial deep search) and the GPU speed-up of training (images preloaded in RAM, batch augmentation on the GPU, mixed precision).
- **Daniel Truzman (100475161):** design of CUSTOM phases 4–5 (widening to 48×2, 60 epochs, selection of n07_60ep, deterministic Kaggle runs) and the CutMix, ensemble, threshold and train+val experiments.
- **Both:** the training runs on both members' machines, the notebook and the fine-tuning search.

Use of AI tools. We used an AI assistant as a productivity tool, always following our own criteria and instructions once we had defined the design criteria ourselves. It helped us implement and debug code, look for errors, suggest possible improvements and edit the text of this report. Every experiment, result and decision was run, checked and taken by the authors.



(a) CUSTOM **(b)** FT (Swin-T)

Figure 1: Validation confusion matrices, row-normalised (%); rows = true class, columns = predicted class.

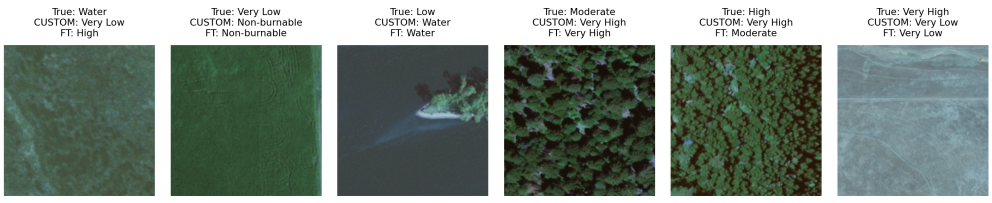


Figure 2: Validation errors of both models: per true class, the shared error with the most confident FT prediction (chosen automatically).

Table 2: Main experiments: validation macro-F1 at the best epoch (L10: mean of the last 10 epochs). Reference and selected models in bold.

Cat.	Change	Objective	Budget	Val F1
CUSTOM	**Reference notebook (StepLR)**	Baseline	60 ep	≈0.36
CUSTOM	GAP instead of Flatten (first version)	Fewer FC parameters; train F1 ≈0.42	30 ep	0.440
CUSTOM	Phases 1–3 random search	Architecture, optimiser, regularisation	15 ep (finalists 40)	0.45–0.465
CUSTOM	Inverse-frequency class weights	Help minority classes	40 ep	L10 0.431 vs 0.453
CUSTOM	48 ch, 2 convs/block (p4_09_B)	More capacity (underfitting)	40 ep	0.496 (L10 0.483)
CUSTOM	64 ch; 5 blocks; crop 288/eval 320	More capacity / resolution	40 ep	no gain
CUSTOM	**60 epochs (n07_60ep)**	Longer schedule	60 ep	**0.502** (L10 0.489)
CUSTOM	CutMix (standard; risk-only)	Regularisation	80 ep	L10 0.483; 0.470 vs 0.486
CUSTOM	Deep search (F3_B_038)	Joint search, 121 trials	80 ep, input 320	0.524
FT	**AlexNet (provided recipe)**	Baseline	5 ep	0.452
FT	ConvNeXt-T (cnx_p2_0)	Modern CNN backbone	30 ep	0.505
FT	ResNet-50 (r50_p2_0)	Residual backbone (train F1 0.99)	30 ep	0.539
FT	**Swin-T (swin_p2_1)**	Transformer backbone	30 ep	**0.546**

Table 3: Class counts and per-class validation precision / recall / F1 of the selected models.

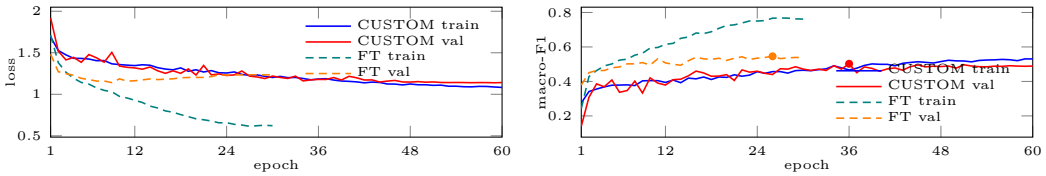
Class	Train	Val	CUSTOM P/R/F1	FT P/R/F1
0 Water	86	29	.83/.86/.85	.81/.86/.83
1 Very Low	1,088	363	.59/.60/.60	.65/.61/.63
2 Low	535	178	.27/.19/.22	.33/.33/.33
3 Moderate	431	144	.29/.35/.32	.39/.37/.38
4 High	315	105	.31/.36/.33	.39/.42/.41
5 Very High	163	54	.41/.39/.40	.41/.44/.43
6 Non-burnable	898	299	.79/.80/.80	.80/.83/.81
Total / macro	3,516	1,172	.500/.508/**.502**	.539/.553/**.546**
Accuracy			.535	.577

Table 4: Public CodaBench macro-F1 (fixed undisclosed test subset). Bold: final submission with the selected models; n/r: not recorded.

ID	Date	Models (custom / ft)	CUSTOM	FT
1036062	05 Nov	p4_09_B / placeholder	0.49	–
1038020	06 Nov	n07, train+val / –	0.49	–
1038186	06 Nov	F3_B_038 / F1_B_037	0.47	0.47
1038323	06 Nov	CUSTOM ens. (5) / + TTA	0.48	n/r
1042782	**08 Nov**	**n07_60ep / swin_p2_1**	**0.50**	**0.49**

Table 5: Final CUSTOM architecture.

Stage	Layers	Output
Input	RGB crop	3 × 224 × 224
Blocks 1–4	[Conv3×3-BN-ReLU]×2, MaxPool 2×2	48/96/192/384 ch
	(spatial size 112/56/28/14)	384 × 14 × 14
GAP	spatial mean	384
Head	FC 384→256, ReLU, Dropout 0.3, FC 256→7	7
2,739,447 parameters; Kaiming initialisation.		



(a) Loss **(b)** Macro-F1

Figure 3: Train (with augmentation) and validation curves of the selected models; dots mark the selected epochs (CUSTOM 36, FT 26).

REFERENCES

[1] Shuchang Shen, Sachith Seneviratne, Xinye Wanyan, et al. FireRisk: A remote sensing dataset for fire risk assessment with benchmarks using supervised and self-supervised learning. In *DICTA*, 2023.

[2] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, et al. mixup: Beyond empirical risk minimization. In *ICLR*, 2018.

[3] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, et al. CutMix: Regularization strategy to train strong classifiers with localizable features. In *ICCV*, 2019.

[4] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.

[5] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.

[6] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. In *ICLR*, 2014.

[7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *NeurIPS*, 2012.

[8] Olga Russakovsky, Jia Deng, Hao Su, et al. ImageNet large scale visual recognition challenge. *IJCV*, 2015.

[9] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, et al. A ConvNet for the 2020s. In *CVPR*, 2022.

[10] Ze Liu, Yutong Lin, Yue Cao, et al. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021.

[11] Mingxing Tan and Quoc V. Le. EfficientNetV2: Smaller models and faster training. In *ICML*, 2021.

[12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, et al. Deep residual learning for image recognition. In *CVPR*, 2016.

[13] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. In *ACL*, 2018.

[14] Gao Huang, Yu Sun, Zhuang Liu, et al. Deep networks with stochastic depth. In *ECCV*, 2016.

[15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[16] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.

[17] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[18] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *JMLR*, 2012.

[19] Daniel Golovin, Benjamin Solnik, Subhodeep Moitra, et al. Google vizier: A service for black-box optimization. In *KDD*, 2017.